Experiment 4

Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

Code :
```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
   pid_t pid1, pid2;

   pid1 = fork();

   if (pid1 == 0)
   {
      printf("Child 1: PID = %d\n", getpid());
      sleep(2);
      printf("Child 1 completed\n");
      return 0;
   }

   pid2 = fork();

   if (pid2 == 0)
   {
      printf("Child 2: PID = %d\n", getpid());
      sleep(3);
      printf("Child 2 completed\n");
      return 0;
   }

   printf("Parent: PID = %d\n", getpid());

   wait(NULL);
   printf("Parent: One child completed using wait()\n");

   waitpid(pid2, NULL, 0);
   printf("Parent: Child 2 completed using waitpid()\n");

   printf("Parent completed\n");

   return 0;
```

}

Out Put:

```
gssgyy@Gyanadeep:~$ nano practical4.c
gssgyy@Gyanadeep:~$ gcc practical4.c -o practical4
gssgyy@Gyanadeep:~$ ./practical4
Parent: PID = 5527
Child 1: PID = 5528
Child 2: PID = 5529
Child 1 completed
Parent: One child completed using wait()
Child 2 completed
Parent: Child 2 completed using waitpid()
Parent completed
gssgyy@Gyanadeep:~$
```